

কীভাবে Hugging Face-এর **Trainer** API-এর ওপর নির্ভর না করে সরাসরি **PyTorch** ব্যবহার করে একটি কাস্টম ট্রেনিং লুপ (**Training Loop**) লেখা যায়। এটি মডেলের প্রতিটি ধাপ নিজের মতো করে নিয়ন্ত্রণ করতে এবং কোনো সমস্যা হলে ডিবাগ করতে সাহায্য করে।

নিচে এর পুরো প্রক্রিয়াটি সহজভাবে বুঝিয়ে বলা হলো:

১. ডেটা প্রস্তুতি (**Data Preparation**)

প্রথমে ডেটাসেটকে প্রসেস করার পর সেগুলোকে **DataLoaders**-এ রূপান্তর করতে হয়।

- **DataLoader**: এটি আপনার ডেটাসেটকে ছোট ছোট ব্যাচে (batches) ভাগ করে মডেলে পাঠানোর জন্য তৈরি করে।
- **DataCollator**: এটি ডাইনামিক প্যাডিং নিশ্চিত করে যাতে প্রতিটি ব্যাচের টেনসরগুলোর শেপ বা আকার সঠিক থাকে,।
- **Shuffling**: ট্রেনিং সেটকে এলোমেলো বা 'shuffle' করা হয় যাতে মডেলটি ভালো শিখতে পারে

২. মডেল এবং অপটিমাইজার (**Model & Optimizer**)

- **Model**: **from_pretrained** মেথড ব্যবহার করে মডেল লোড করা হয় এবং আপনার ডেটাসেটের ক্লাসের সংখ্যা অনুযায়ী এর লেবেল ঠিক করা হয়।
- **Optimizer**: মডেলের ওয়েট (weights) আপডেট করার জন্য এখানে **AdamW** অপটিমাইজার ব্যবহার করা হয়েছে। এটি লস (loss) কমানোর জন্য মডেলের প্রতিটি ওয়েটকে সামান্য পরিবর্তন করে দেয়,।
- **Zero Gradient**: প্রতিটি স্টেপ শেষে গ্রেডিয়েন্টগুলো জিরো করে দেওয়া জরুরি, নাহলে পরের স্টেপে সেগুলো আগেরগুলোর সাথে যোগ হয়ে যাবে

৩. লার্নিং রেট এবং ডিভাইস (**Scheduler & Device**)

- **Scheduler**: একটি লার্নিং রেট শিডিউলার যোগ করা হয় যা ট্রেনিং চলাকালীন ধীরে ধীরে লার্নিং রেট কমিয়ে শূন্যের দিকে নিয়ে আসে।
- **GPU (CUDA)**: ট্রেনিংয়ের গতি কয়েক ঘন্টা থেকে কমিয়ে কয়েক মিনিটে নিয়ে আসার জন্য মডেল এবং ডেটাকে **GPU** বা CUDA ডিভাইসে পাঠানো হয়,

৪. ট্রেনিং লুপের ধাপসমূহ (**The Training Loop**)

ট্রেনিং শুরু করার জন্য মডেলকে প্রথমে **model.train()** মোডে নিতে হয়। এরপর প্রতিটি ব্যাচের জন্য নিচের ধাপগুলো অনুসরণ করা হয়: ১. ডেটাগুলো GPU-তে পাঠানো হয়। ২. মডেল থেকে আউটপুট এবং **Loss** বের করা হয়। ৩. **loss.backward()** ব্যবহার করে গ্রেডিয়েন্ট হিসেব করা হয় ৪. **optimizer.step()** দিয়ে মডেলের ওয়েট আপডেট করা হয়। ৫. **scheduler.step()** দিয়ে লার্নিং রেট আপডেট করা হয় এবং অপটিমাইজারের গ্রেডিয়েন্ট জিরো করা হয়।

৫. ইভ্যালুয়েশন (**Evaluation**)

ট্রেনিং শেষ হওয়ার পর মডেলটি কেমন কাজ করছে তা যাচাই করা হয়:

- মডেলকে `model.eval()` মোডে নেওয়া হয়।
- মডেল থেকে পাওয়া **Logits**-এর ওপর `argmax` ফাংশন চালিয়ে সেগুলোকে প্রডিকশনে রূপান্তর করা হয়।
- সবশেষে `Datasets` লাইব্রেরির `metric.compute()` মেথড ব্যবহার করে চূড়ান্ত ফলাফল (যেমন Accuracy) বের করা হয়।

সংক্ষেপে, এই পদ্ধতিতে আপনি আপনার মডেল ট্রেনিংয়ের প্রতিটি ছোট ছোট বিষয় (যেমন লস ক্যালকুলেশন, গ্রেডিয়েন্ট আপডেট) নিজে কোড লিখে নিয়ন্ত্রণ করতে পারেন।

